

Lecture 2 - Kinematics and Behavioral robotics

Mobile Robotics (ELA408)

Lennie Carlén Eriksson

Mälardalen University

lennie.carlen.eriksson@mdu.se

Version: April 1, 2026

Introduction – Outline

1. Introduction
2. Kinematic models of mobile robots
3. Odometry
4. Behavior-based Robotics
5. Behaviors revisited

Introduction

Mobile Robot Kinematics: Aim

- Description of mechanical behavior of the robot for **design and control**
- Similar to robot manipulator kinematics

Mobile Robot Kinematics: Aim

- Description of mechanical behavior of the robot for **design and control**
- Similar to robot manipulator kinematics
- However, mobile robots can move unbound with respect to their environment

Mobile Robot Kinematics: Aim

- Description of mechanical behavior of the robot for **design and control**
- Similar to robot manipulator kinematics
- However, mobile robots can move unbound with respect to their environment
 - There is **no direct way to measure the robot's position**

Mobile Robot Kinematics: Aim

- Description of mechanical behavior of the robot for **design and control**
- Similar to robot manipulator kinematics
- However, mobile robots can move unbound with respect to their environment
 - There is **no direct way to measure the robot's position**
 - **Position must be integrated over time**

Mobile Robot Kinematics: Aim

- Description of mechanical behavior of the robot for **design and control**
- Similar to robot manipulator kinematics
- However, mobile robots can move unbound with respect to their environment
 - There is **no direct way to measure the robot's position**
 - **Position must be integrated over time**
 - Leads to inaccuracies of the position (motion) estimate

Mobile Robot Kinematics: Aim

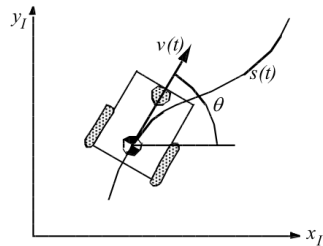
- Description of mechanical behavior of the robot for **design and control**
- Similar to robot manipulator kinematics
- However, mobile robots can move unbound with respect to their environment
 - There is **no direct way to measure the robot's position**
 - **Position must be integrated over time**
 - Leads to inaccuracies of the position (motion) estimate
 - **the number one challenge in mobile robotics**

Mobile Robot Kinematics: Aim

- Description of mechanical behavior of the robot for **design and control**
- Similar to robot manipulator kinematics
- However, mobile robots can move unbound with respect to their environment
 - There is **no direct way to measure the robot's position**
 - **Position must be integrated over time**
 - Leads to inaccuracies of the position (motion) estimate
 - **the number one challenge in mobile robotics**
- Understanding mobile robot motion starts with understanding **wheel constraints** placed on the robots mobility

Mobile Robot Kinematics: Goal

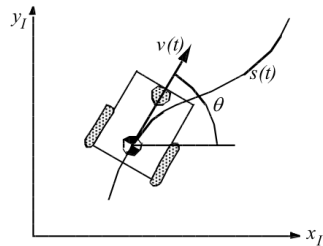
- Establish the robot speed $\dot{\xi} = [\dot{x} \quad \dot{y} \quad \dot{\theta}]^T$ as a function of the wheel speeds $\dot{\phi}_i$, steering angles γ_i , steering speeds $\dot{\gamma}_i$ and geometric parameters of the robot (**configuration coordinates**).



Mobile Robot Kinematics: Goal

- Establish the robot speed $\dot{\xi} = [\dot{x} \quad \dot{y} \quad \dot{\theta}]^T$ as a function of the wheel speeds $\dot{\phi}_i$, steering angles γ_i , steering speeds $\dot{\gamma}_i$ and geometric parameters of the robot (**configuration coordinates**).
- Forward kinematics**

$$\dot{\xi} = \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \mathbf{f}(\dot{\phi}_1, \dots, \dot{\phi}_n, \gamma_1, \dots, \gamma_m, \dot{\gamma}_1, \dots, \dot{\gamma}_m)$$



Mobile Robot Kinematics: Goal

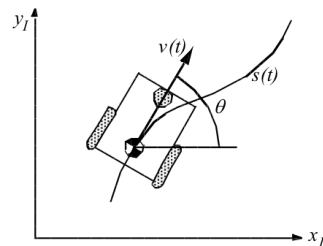
- Establish the robot speed $\dot{\xi} = [\dot{x} \quad \dot{y} \quad \dot{\theta}]^T$ as a function of the wheel speeds $\dot{\phi}_i$, steering angles γ_i , steering speeds $\dot{\gamma}_i$ and geometric parameters of the robot (**configuration coordinates**).

- Forward kinematics**

$$\dot{\xi} = \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \mathbf{f}(\dot{\phi}_1, \dots, \dot{\phi}_n, \gamma_1, \dots, \gamma_m, \dot{\gamma}_1, \dots, \dot{\gamma}_m)$$

- Inverse kinematics**

$$[\dot{\phi}_1 \quad \dots \quad \dot{\phi}_n \quad \gamma_1 \quad \dots \quad \gamma_m \quad \dot{\gamma}_1 \quad \dots \quad \dot{\gamma}_m]^T = \mathbf{g}(\dot{x}, \dot{y}, \dot{\theta})$$



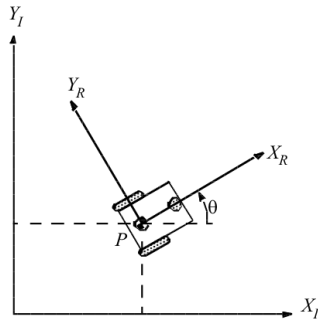
Representing Robot Position

- Representing a robot within an arbitrary initial frame

- Initial frame: $\{X_I, Y_I\}$
- Robot frame: $\{X_R, Y_R\}$
- Robot position $\xi_I = [x \ y \ \theta]^\top$
- Mapping between the two frames

$$\dot{\xi}_R = \mathbf{R}(\theta)\dot{\xi}_I = \mathbf{R}(\theta) \begin{bmatrix} \dot{x} & \dot{y} & \dot{\theta} \end{bmatrix}^\top$$

$$\mathbf{R}(\theta) = \begin{bmatrix} \cos \theta & \sin \theta & 0 \\ -\sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$



Representing Robot Position

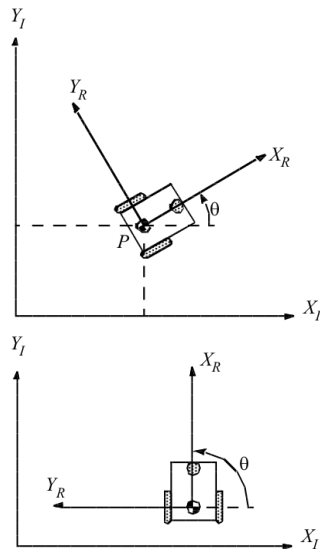
- Representing a robot within an arbitrary initial frame

- Initial frame: $\{X_I, Y_I\}$
- Robot frame: $\{X_R, Y_R\}$
- Robot position $\xi_I = [x \ y \ \theta]^T$
- Mapping between the two frames

$$\dot{\xi}_R = \mathbf{R}(\theta)\dot{\xi}_I = \mathbf{R}(\theta)[\dot{x} \ \dot{y} \ \dot{\theta}]^T$$

$$\mathbf{R}(\theta) = \begin{bmatrix} \cos \theta & \sin \theta & 0 \\ -\sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

- Example: Robot aligned with Y_I

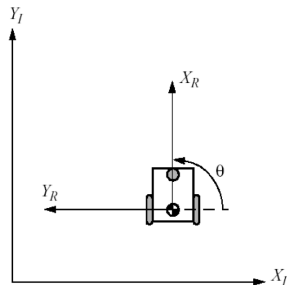


Example

$$\mathbf{R}(\theta) = \begin{bmatrix} \cos \theta & \sin \theta & 0 \\ -\sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

and

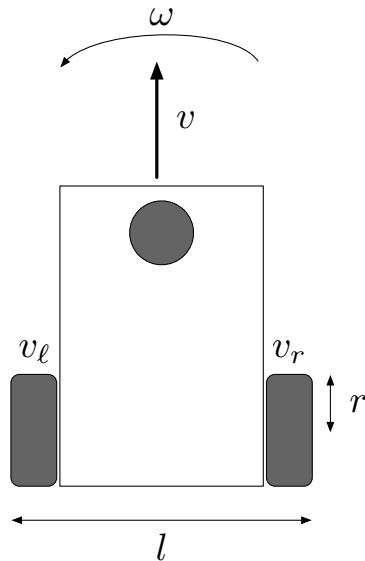
$$\dot{\xi}_R = \mathbf{R}\left(\frac{\pi}{2}\right) \dot{\xi}_I = \begin{bmatrix} 0 & 1 & 0 \\ -1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} \dot{y} \\ -\dot{x} \\ \dot{\theta} \end{bmatrix}$$



Kinematic models of mobile robots

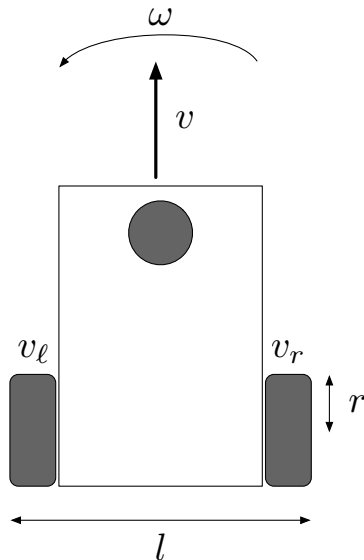
Differential drive

- Construction:
 - 2 wheels on the same axis
 - 2 motors (one for wheel)
 - Third passive supporting wheel
 - Wheel radius: r
 - Wheel base: l



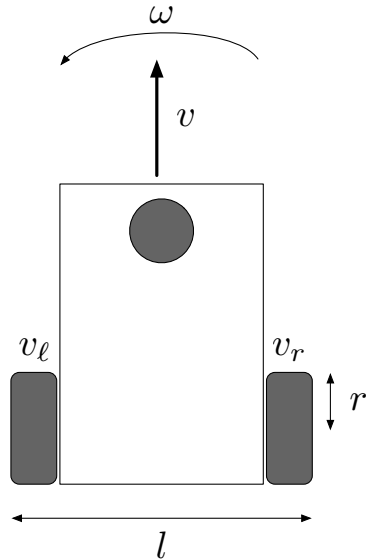
Differential drive

- Construction:
 - 2 wheels on the same axis
 - 2 motors (one for wheel)
 - Third passive supporting wheel
 - Wheel radius: r
 - Wheel base: l
- Control input
 - v : linear velocity of the robot
 - ω : angular velocity of the robot



Differential drive

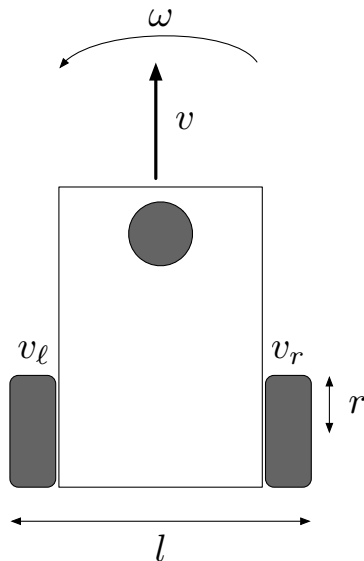
- Construction:
 - 2 wheels on the same axis
 - 2 motors (one for wheel)
 - Third passive supporting wheel
 - Wheel radius: r
 - Wheel base: l
- Control input
 - v : linear velocity of the robot
 - ω : angular velocity of the robot
- State
 - (x, y) position
 - θ orientation of the robot



Differential drive

- Construction:
 - 2 wheels on the same axis
 - 2 motors (one for wheel)
 - Third passive supporting wheel
 - Wheel radius: r
 - Wheel base: l
- **Control input**
 - v : linear velocity of the robot
 - ω : angular velocity of the robot
- **State**
 - (x, y) position
 - θ orientation of the robot
- 2 variables independently controlled (**actuation**)
 - v_r : velocity of the right wheel
 - v_l : velocity of the left wheel

Kinematic models of mobile robots

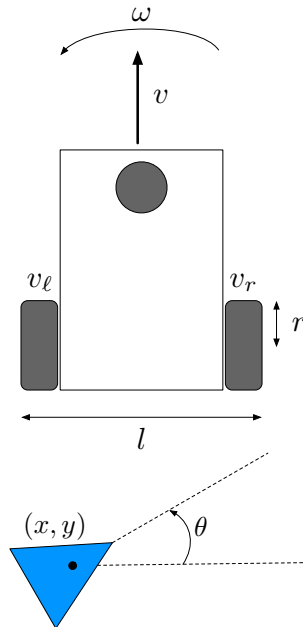


Differential drive: Dynamic model

$$\begin{cases} \dot{x} = \frac{r}{2}(v_r + v_\ell) \cos \theta \\ \dot{y} = \frac{r}{2}(v_r + v_\ell) \sin \theta \\ \dot{\theta} = \frac{r}{l}(v_r - v_\ell) \end{cases}$$

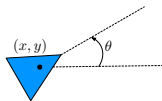
This is a model

- Not very used for design control strategies
- Very useful for implementing the control strategy



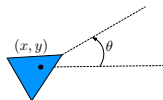
The “Unicycle” Model

- But it is not very natural to “think” in terms of wheels velocities



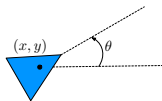
The “Unicycle” Model

- But it is not very natural to “think” in terms of wheels velocities
- Go directly for translational and angular velocities



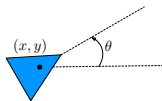
The “Unicycle” Model

- But it is not very natural to “think” in terms of wheels velocities
 - Go directly for translational and angular velocities
- Inputs:
 - v : forward velocity
 - ω : angular velocity



The “Unicycle” Model

- But it is not very natural to “think” in terms of wheels velocities
- Go directly for translational and angular velocities



- Inputs:
 - v : forward velocity
 - ω : angular velocity
- Dynamics:

$$\begin{cases} \dot{x} = v \cos \theta \\ \dot{y} = v \sin \theta \\ \dot{\theta} = \omega \end{cases}$$

How do we match the differential drive model with the unicycle model?

- We design for this model

$$\begin{cases} \dot{x} = v \cos \theta \\ \dot{y} = v \sin \theta \\ \dot{\theta} = \omega \end{cases}$$

- We have to implement our system on this model:

$$\begin{cases} \dot{x} = \frac{r}{2}(v_r + v_\ell) \cos \theta \\ \dot{y} = \frac{r}{2}(v_r + v_\ell) \sin \theta \\ \dot{\theta} = \frac{r}{l}(v_r - v_\ell) \end{cases}$$

How do we match the differential drive model with the unicycle model?

- We design for this model

$$\begin{cases} \dot{x} = v \cos \theta \\ \dot{y} = v \sin \theta \\ \dot{\theta} = \omega \end{cases}$$

- We have to implement our system on this model:

$$\begin{cases} \dot{x} = \frac{r}{2}(v_r + v_\ell) \cos \theta \\ \dot{y} = \frac{r}{2}(v_r + v_\ell) \sin \theta \\ \dot{\theta} = \frac{r}{l}(v_r - v_\ell) \end{cases}$$

- The connection is the following:

$$\begin{aligned} v &= \frac{r}{2}(v_r + v_\ell) \quad \Rightarrow \quad \frac{2v}{r} = v_r + v_\ell \\ \omega &= \frac{r}{l}(v_r - v_\ell) \quad \Rightarrow \quad \frac{\omega l}{r} = v_r - v_\ell \end{aligned}$$

How do we match the differential drive model with the unicycle model?

- We design for this model

$$\begin{cases} \dot{x} = v \cos \theta \\ \dot{y} = v \sin \theta \\ \dot{\theta} = \omega \end{cases}$$

- We have to implement our system on this model:

$$\begin{cases} \dot{x} = \frac{r}{2}(v_r + v_\ell) \cos \theta \\ \dot{y} = \frac{r}{2}(v_r + v_\ell) \sin \theta \\ \dot{\theta} = \frac{r}{l}(v_r - v_\ell) \end{cases}$$

- The connection is the following:

$$\begin{aligned} v = \frac{r}{2}(v_r + v_\ell) &\Rightarrow \frac{2v}{r} = v_r + v_\ell \\ \omega = \frac{r}{l}(v_r - v_\ell) &\Rightarrow \frac{\omega l}{r} = v_r - v_\ell \end{aligned}$$

- That we can solve for v_r and v_ℓ :

$$\begin{aligned} v_r &= \frac{2v + \omega l}{2r} \\ v_\ell &= \frac{2v - \omega l}{2r} \end{aligned}$$

Odometry

Odometry

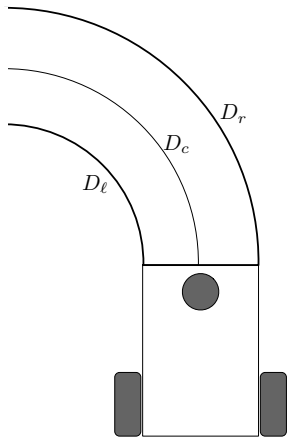
- The state of the robot is (x, y, θ)
- How do we obtain this state information?

Odometry

- The state of the robot is (x, y, θ)
- How do we obtain this state information?
- Sensors
 - **External sensors**, measuring something in the environment
 - Cameras, LIDAR, Ultrasound, Infrared, GPS (not enough for hi-level of fidelity), ...
 - **Internal sensors**, “sitting” in the robot
 - Orientation: Compass
 - Position: accelerometers and gyroscopes
 - **Wheel Encoders**

Wheel Encoder

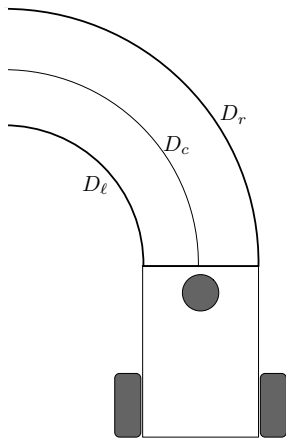
- Wheel encoders give the distance moved by each wheel
- Assume that the wheels are following an arc (short time scale)



$$D_c = \frac{D_\ell + D_r}{2}$$

Wheel Encoder

- Wheel encoders give the distance moved by each wheel
- Assume that the wheels are following an arc (short time scale)

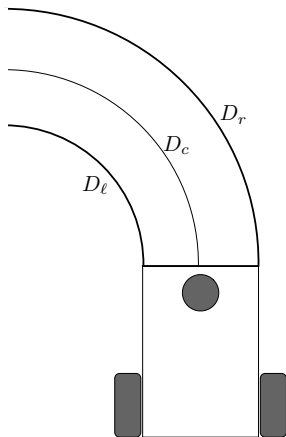


$$D_c = \frac{D_\ell + D_r}{2}$$

$$\begin{cases} x' = x + D_c \cos \theta \\ y' = y + D_c \sin \theta \\ \theta' = \theta + \frac{D_r - D_\ell}{l} \end{cases}$$

Wheel Encoder

- Wheel encoders give the distance moved by each wheel
- Assume that the wheels are following an arc (short time scale)



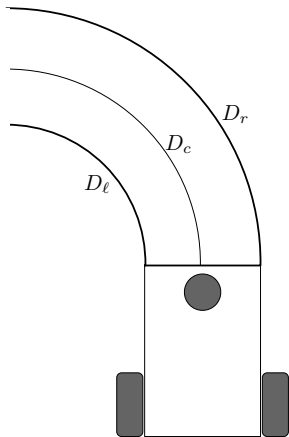
$$D_c = \frac{D_\ell + D_r}{2}$$

$$\begin{cases} x' = x + D_c \cos \theta \\ y' = y + D_c \sin \theta \\ \theta' = \theta + \frac{D_r - D_\ell}{l} \end{cases}$$

How do we know D_r and D_ℓ ?

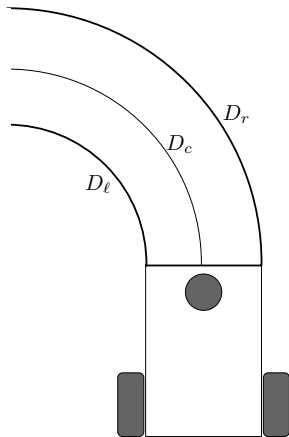
Wheel Encoder

- How do we know how far each wheel has moved?
 - Assume that each wheel has N ticks per revolution
 - Most of the wheel encoders give the total tick count since the beginning



Wheel Encoder

- How do we know how far each wheel has moved?
 - Assume that each wheel has N ticks per revolution
 - Most of the wheel encoders give the total tick count since the beginning
 - For both wheels

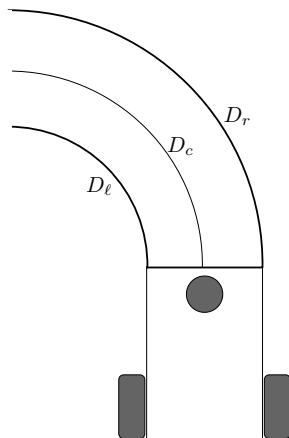


$$\Delta \text{tick}_r = \text{tick}'_r - \text{tick}_r$$

$$\Delta \text{tick}_\ell = \text{tick}'_\ell - \text{tick}_\ell$$

Wheel Encoder

- How do we know how far each wheel has moved?
 - Assume that each wheel has N ticks per revolution
 - Most of the wheel encoders give the total tick count since the beginning
 - For both wheels



$$\Delta \text{tick}_r = \text{tick}'_r - \text{tick}_r$$

$$\Delta \text{tick}_\ell = \text{tick}'_\ell - \text{tick}_\ell$$

- Based on the ticks one can compute the total distance

$$D_r = 2\pi r \frac{\Delta \text{tick}_r}{N}$$

$$D_\ell = 2\pi r \frac{\Delta \text{tick}_\ell}{N}$$

Warning: Drift!

- Using wheel encoders as the only source of odometry
 - Very imprecise
 - Can drift due to inaccuracies in the measurement
 - Can drift very much when the wheel has no contact with the ground
 - Not particularly robust

Behavior-based Robotics

Behavior-based robotics

- The world is fundamentally unknown and dynamic
- Does not make sense to over-plan
- Key idea: Develop a library of useful controllers (behaviors)
- Switch among controllers in response to environmental changes

Building a behavior

- Assume that you have a differential-drive, wheeled mobile robot driving at constant speed

$$\begin{cases} \dot{x} = v_0 \cos \theta \\ \dot{y} = v_0 \sin \theta \\ \dot{\theta} = \omega \end{cases}$$

Building a behavior

- Assume that you have a differential-drive, wheeled mobile robot driving at constant speed

$$\begin{cases} \dot{x} = v_0 \cos \theta \\ \dot{y} = v_0 \sin \theta \\ \dot{\theta} = \omega \end{cases}$$

- How do we steer?

Building a behavior

- Assume that you have a differential-drive, wheeled mobile robot driving at constant speed

$$\begin{cases} \dot{x} = v_0 \cos \theta \\ \dot{y} = v_0 \sin \theta \\ \dot{\theta} = \omega \end{cases}$$

- How do we steer?
- We want to steer the robot to a desired orientation θ_d
- We have:
 - A reference θ_d
 - A control input ω
 - A tracking error $e = \theta_d - \theta$
 - A model $\dot{\theta} = \omega$

Building a Behavior

- We can think of using a PID controller

$$\omega = K_P e + K_I \int e d\tau + K_D \frac{de}{dt}$$

- Will it work like this??

Building a Behavior

- We can think of using a PID controller

$$\omega = K_P e + K_I \int e d\tau + K_D \frac{de}{dt}$$

- Will it work like this??
- Not really since we are dealing with angles
 - For example: $\theta_d = 0, \theta = 100\pi, \Rightarrow e = -100\pi$

Building a Behavior

- We can think of using a PID controller

$$\omega = K_P e + K_I \int e d\tau + K_D \frac{de}{dt}$$

- Will it work like this??
- Not really since we are dealing with angles
 - For example: $\theta_d = 0, \theta = 100\pi, \Rightarrow e = -100\pi$
- Solution is to ensure that $e \in [-\pi, \pi]$

Building a Behavior

- We can think of using a PID controller

$$\omega = K_P e + K_I \int e d\tau + K_D \frac{de}{dt}$$

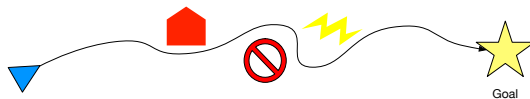
- Will it work like this??
- Not really since we are dealing with angles
 - For example: $\theta_d = 0, \theta = 100\pi, \Rightarrow e = -100\pi$
- Solution is to ensure that $e \in [-\pi, \pi]$
- Standard trick: use the *atan2* function

$$e' = \text{atan2}(\sin(e), \cos(e)) \in [-\pi, \pi]$$

and with this simple caveat the PID controller will work fine (if correctly tuned)

Navigation

- Problem: Going to a goal location without bumping into obstacles



- At least we need two behaviors
 1. Go-to-goal
 2. Avoid-obstacles

Go-to-goal behavior

- How to drive a robot to a goal location?
- We have the following:

$$\begin{cases} \dot{x} = v_0 \cos \theta \\ \dot{y} = v_0 \sin \theta \\ \dot{\theta} = \omega \end{cases} \quad e = \theta_d - \theta \quad \omega = \text{PID}(e)$$

Go-to-goal behavior

- How to drive a robot to a goal location?
- We have the following:

$$\begin{cases} \dot{x} = v_0 \cos \theta \\ \dot{y} = v_0 \sin \theta \\ \dot{\theta} = \omega \end{cases} \quad e = \theta_d - \theta \quad \omega = \text{PID}(e)$$

- The question is
 - We start from a position (x, y)
 - We want to reach a position (x_g, y_g)

Go-to-goal behavior

- How to drive a robot to a goal location?
- We have the following:

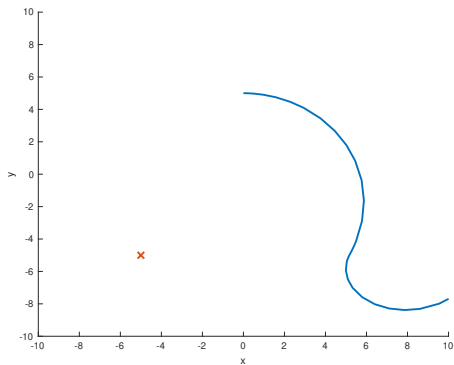
$$\begin{cases} \dot{x} = v_0 \cos \theta \\ \dot{y} = v_0 \sin \theta \\ \dot{\theta} = \omega \end{cases} \quad e = \theta_d - \theta \quad \omega = \text{PID}(e)$$

- The question is
 - We start from a position (x, y)
 - We want to reach a position (x_g, y_g)
 - We can compute the desired orientation as

$$\theta_d = \text{atan2} \left(\frac{y_g - y}{x_g - x} \right)$$

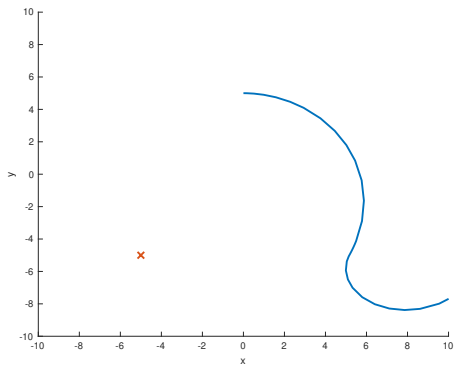
Attempt 1

$$\omega = K(\theta_d - \theta)$$



Attempt 1

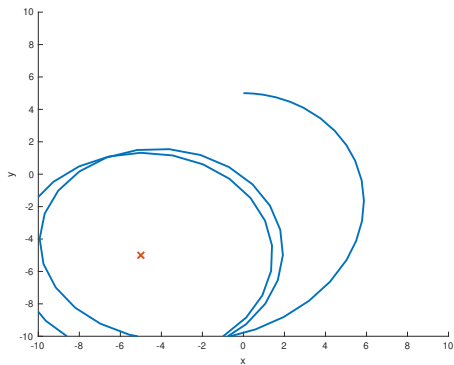
$$\omega = K(\theta_d - \theta)$$



We forgot to add the $\text{atan2}(e)$!

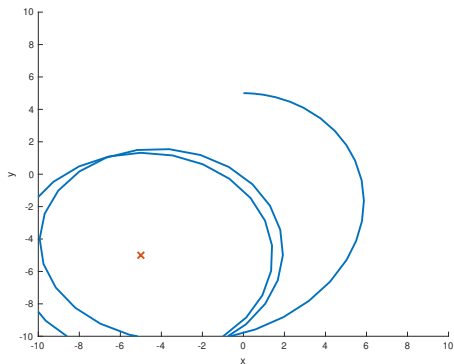
Attempt 2

$$\omega = K(\theta_d - \theta)$$



Attempt 2

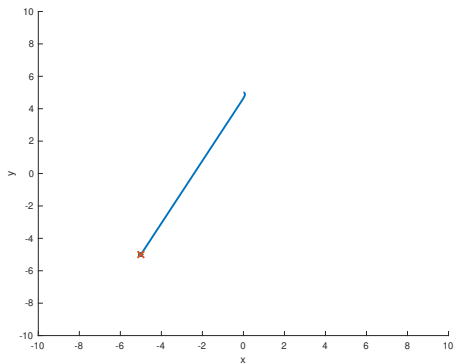
$$\omega = K(\theta_d - \theta)$$



The proportional gain is too low

Attempt 3

$$\omega = K(\theta_d - \theta)$$



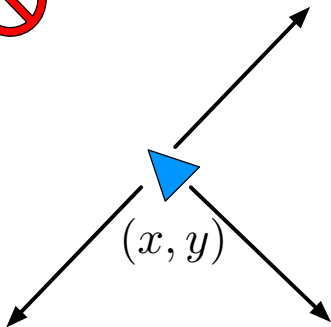
Behavior-based Robotics

Obstacle avoidance

Obstacle avoidance

- How avoid driving into obstacles?
- We can use the same idea by defining a desired heading

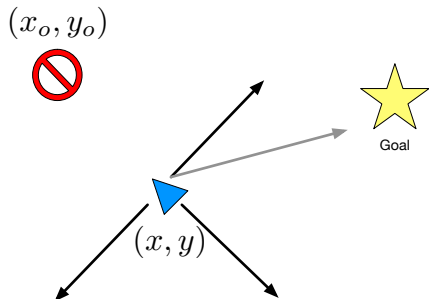
(x_o, y_o)



Which direction should we go?

Obstacle avoidance

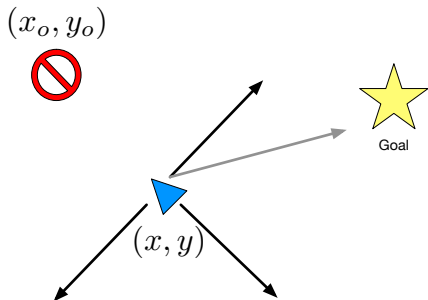
- How avoid driving into obstacles?
- We can use the same idea by defining a desired heading



Can we combine the information related to the goal with the obstacle avoidance?

Obstacle avoidance

- How avoid driving into obstacles?
- We can use the same idea by defining a desired heading



Can we combine the information related to the goal with the obstacle avoidance?

- There is no obvious “correct” answer
- Some choices can be better than others based on the situation

Where to go? – Pure avoidance strategy



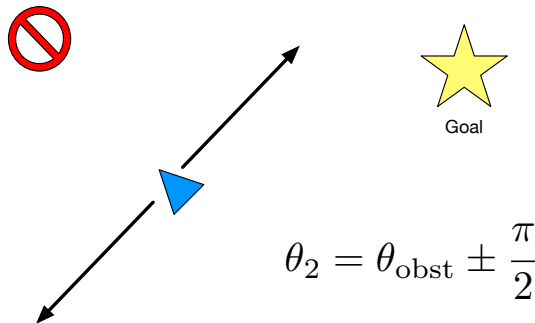
Goal



$$\theta_1 = \theta_{\text{obst}} + \pi$$

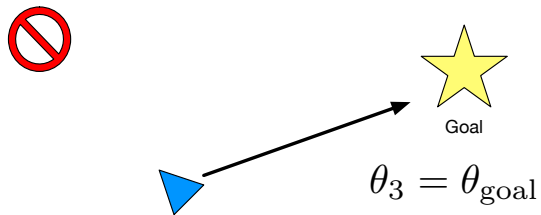
- With $\theta_{\text{obst}} = \text{atan2} \left(\frac{y_o - y}{x_o - x} \right)$
- Adding or subtracting π does not matter in this case

Where to go? – Blended avoidance strategy



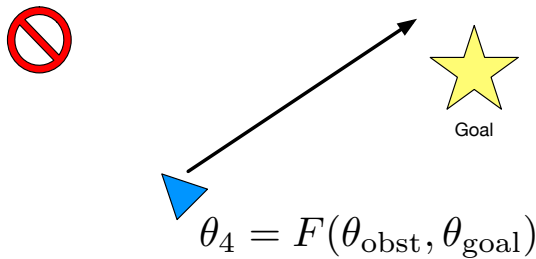
- Adding or subtracting $\pi/2$ really matters
- Choice depends on the direction of the goal:
 - Not “pure” but **blended**
 - We can choose based on where the goal is located, e.g., $+\pi/2$ since it goes in the direction of the goal

Where to go? – Pure go-to-goal strategy



- Pure go-to-goal

Where to go? – Blended strategy



- Combination of the direction to the goal and the direction of the obstacle

Arbitration Mechanisms

- This example illustrate two fundamentally different **arbitration mechanisms**
 - **Winner takes all** ⇒ Hard switches
 - **Blending** ⇒ Combined behaviors
- Both approaches have merit in different situations
 - Pure strategies are usually easier to **analyze**
 - Blended strategies usually **perform** better

Behaviors revisited

Start simple: The model

- For the purpose of **planning**, assume that $\mathbf{u}$ is the forward velocity

$$\dot{\mathbf{x}} = \mathbf{u}, \quad \mathbf{x} \in \mathbb{R}^2, \mathbf{u} \in \mathbb{R}^2$$

Start simple: The model

- For the purpose of **planning**, assume that $\mathbf{u}$ is the forward velocity

$$\dot{\mathbf{x}} = \mathbf{u}, \quad \mathbf{x} \in \mathbb{R}^2, \mathbf{u} \in \mathbb{R}^2$$

- The model is then:

$$\dot{\mathbf{x}} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix} \mathbf{x} + \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \mathbf{u}$$

Start simple: The model

- For the purpose of **planning**, assume that $\mathbf{u}$ is the forward velocity

$$\dot{\mathbf{x}} = \mathbf{u}, \quad \mathbf{x} \in \mathbb{R}^2, \mathbf{u} \in \mathbb{R}^2$$

- The model is then:

$$\dot{\mathbf{x}} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix} \mathbf{x} + \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \mathbf{u}$$

- The controllability matrix

$$\mathbf{W}_c = [\mathbf{B} \quad \mathbf{AB}] = [\mathbf{I} \quad \mathbf{0}], \quad \text{rank}(\mathbf{W}_c) = 2$$

The system is **completely controllable**!

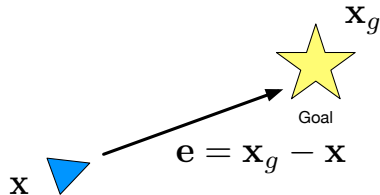
The dynamic duo

- The two-bread-and-butter behaviors are go-to-goal and avoid-obstacle
 - **Go-to-goal**: Drives the robot towards the goal
 - **Avoid-obstacle**: Don't slam into things

The dynamic duo

- The two-bread-and-butter behaviors are go-to-goal and avoid-obstacle
 - **Go-to-goal**: Drives the robot towards the goal
 - **Avoid-obstacle**: Don't slam into things
- Control design task: pick a desired motion vector and set that equal to the input $\mathbf{u}$

Go-to-goal



- The current position is $\mathbf{x}$ and the goal position is $\mathbf{x}_g$, then

$$\mathbf{e} = \mathbf{x}_g - \mathbf{x}$$

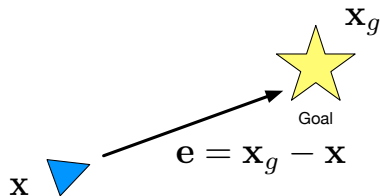
- We can compute $\mathbf{u}$ as

$$\mathbf{u} = \mathbf{K}\mathbf{e} \Rightarrow \dot{\mathbf{e}} = \mathbf{0} - \dot{\mathbf{x}} = -\mathbf{K}\mathbf{e}$$

- To make the system asymptotically stable we have to pick
 - $K > 0$ or $\mathbf{K} > 0$ ($\text{eig}\{\mathbf{K}\} > 0$)
 - $\mathbf{e} \rightarrow 0$

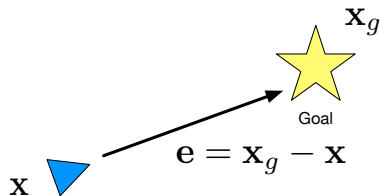
Go-to-goal: A concern

- A linear controller makes the robot go faster the further away the goal is.



Go-to-goal: A concern

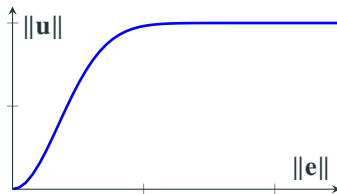
- A linear controller makes the robot go faster the further away the goal is.
- Solution, in practice, make the gain K a function of $\mathbf{e}$



$$\dot{\mathbf{e}} = -K(\|\mathbf{e}\|)\mathbf{e}$$

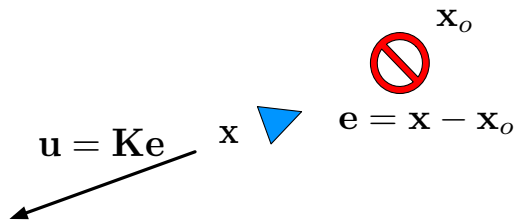
■ For example

$$K = \frac{v_0(1 - e^{-\alpha\|\mathbf{e}\|^2})}{\|\mathbf{e}\|}$$



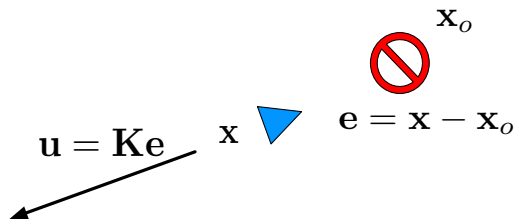
Avoid-Obstacle

- To get the direction, just flip the sign in the go-to-goal behavior



Avoid-Obstacle

- To get the direction, just flip the sign in the go-to-goal behavior



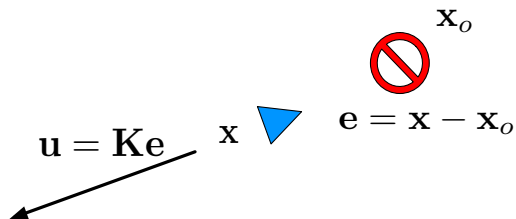
- By this choice

$$\mathbf{u} = \mathbf{K}\mathbf{e} \Rightarrow \dot{\mathbf{e}} = \dot{\mathbf{x}} - \mathbf{0} = \mathbf{K}\mathbf{e}$$

Unstable system!

Avoid-Obstacle

- To get the direction, just flip the sign in the go-to-goal behavior



- By this choice

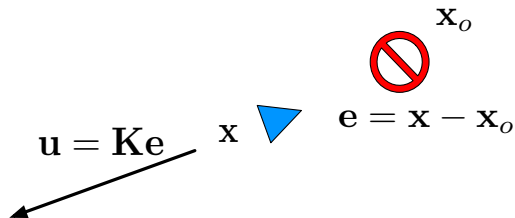
$$u = Ke \Rightarrow \dot{e} = \dot{x} - 0 = Ke$$

Unstable system!

- We don't want the error to converge to 0, so we want it to be "unstable"... but not too much...

Avoid-Obstacle: More concerns

- The robot drives off to infinity?
- It is overly cautious?
- We care less the closer we get?



Avoid-Obstacle: More concerns

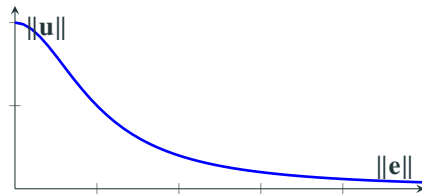
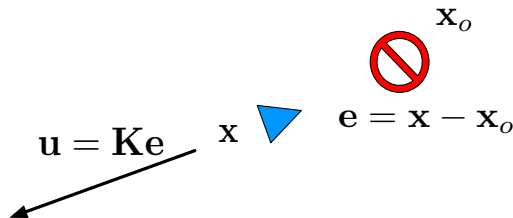
- The robot drives off to infinity?
- It is overly cautious?
- We care less the closer we get?

- **Solutions**

- Make K dependent on $\mathbf{e}$
 - For example

$$K = \frac{1}{\|\mathbf{e}\|} \left(\frac{c}{\|\mathbf{e}\|^2 + \epsilon} \right)$$

- Switch between behaviors
- Use the **induced mode**



Behavior/Mode transitions?

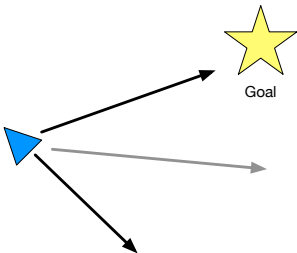
- What's missing?
 - Couple actual robot dynamics to the behaviors
 - Figure out the mode transitions

Behaviors revisited

Hard switches vs Blending

Hard switches vs Blending

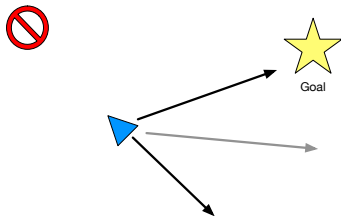
- Given two different behaviors, how should they be combined?



Two options

- **Hard switches**
- **Blended behaviors**

Pros and Cons



“Can robots chew gum and walk at the same time?”

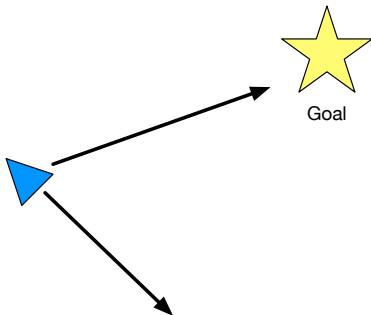
Hard switches

- Pro: Performance guarantees
- Cons
 - Bumpy riding
 - Zeno behavior (infinitely quickly switching between the two modes)

Blending

- Pro: Smooth ride
- Con: No guarantees

A switch Hybrid Automaton



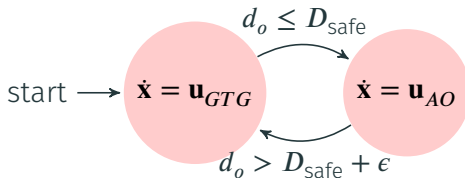
The system is just

$$\dot{\mathbf{x}} = \mathbf{u}$$

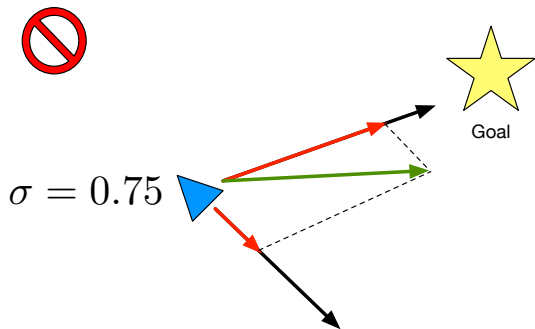
and we have

$$\mathbf{u}_{GTG} = K_{GTG}(\mathbf{x}_g - \mathbf{x})$$

$$\mathbf{u}_{AO} = K_{AO}(\mathbf{x} - \mathbf{x}_o)$$



Blending



- Blending function

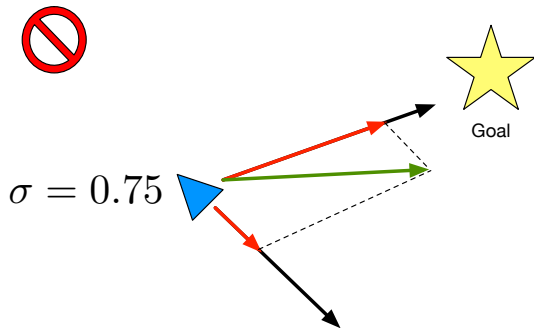
$$\sigma(d_o) \in [0, 1]$$

with 0 only avoid obstacles, while 1 is we only go to the goal

- The system then becomes:

$$\dot{\mathbf{x}} = \sigma(d_o)\mathbf{u}_{GTG} + (1 - \sigma(d_o))\mathbf{u}_{AO}$$

Blending



- Blending function

$$\sigma(d_o) \in [0, 1]$$

with 0 only avoid obstacles, while 1 is we only go to the goal

- The system then becomes:

$$\dot{\mathbf{x}} = \sigma(d_o)\mathbf{u}_{GTG} + (1 - \sigma(d_o))\mathbf{u}_{AO}$$

- Design example of the blending function:

$$\sigma = 1 - e^{-\beta d_o}$$

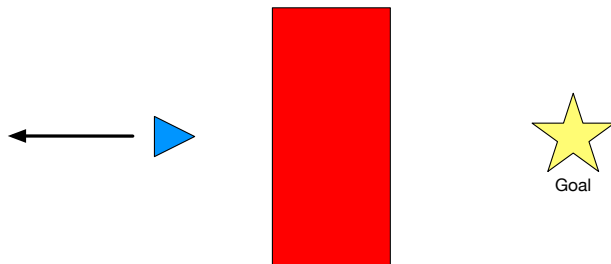
Summary

- Two choices – both of them come with pros and cons
- Would like to have our cake and eat it (smooth ride + performance guarantees)
- Enter: the induced mode

Behaviors revisited

Boundary following

What about this case?



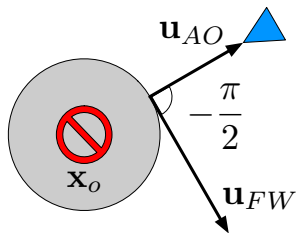
Neither the go-to-goal nor the avoid obstacle mode are useful...

Wall following

- We need some way of negotiating complex environments
- A really useful behavior is one that makes the robot **follow the boundary of an obstacle/wall**
- This will also allow to introduce the **induced mode** in a systematic way

How to follow walls

- The follow-wall behavior should maintain a constant distance to the obstacle/wall it is following



$$\mathbf{u}_{FW} = \alpha \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix} \mathbf{u}_{AO} = \alpha \mathbf{R} \left(-\frac{\pi}{2} \right) \mathbf{u}_{AO}$$

Clockwise or Counter-Clockwise?

- We can clearly move in two different directions along a wall!

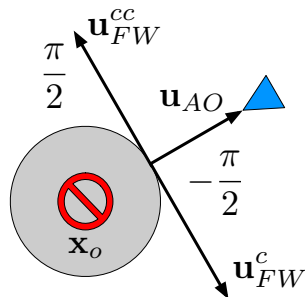
Rotation matrix

$$\mathbf{R}(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$$

Therefore we have:

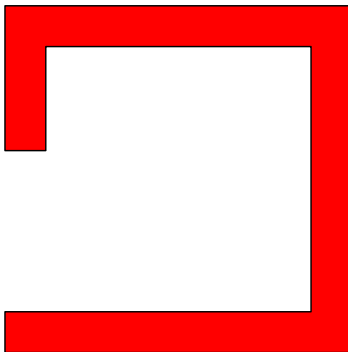
$$\mathbf{u}_{FW}^c = \alpha \mathbf{R}\left(-\frac{\pi}{2}\right) \mathbf{u}_{AO} = \alpha \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix} \mathbf{u}_{AO}$$

$$\mathbf{u}_{FW}^{cc} = \alpha \mathbf{R}\left(\frac{\pi}{2}\right) \mathbf{u}_{AO} = \alpha \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \mathbf{u}_{AO}$$



Clockwise or Counter-Clockwise?

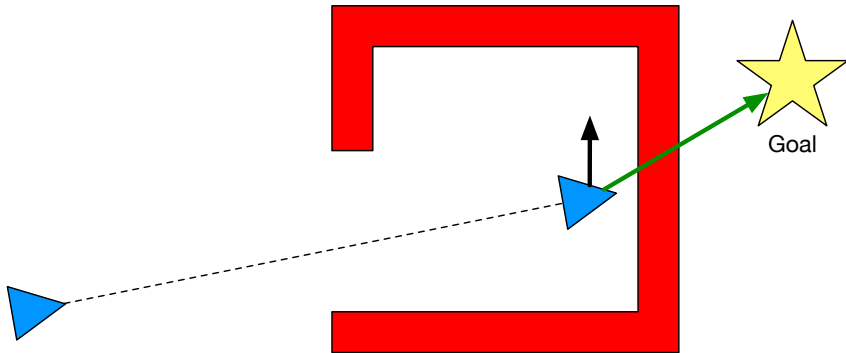
- Which direction to choose?



Goal

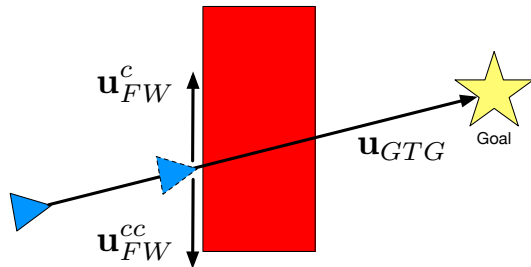
Clockwise or Counter-Clockwise?

- Which direction to choose?



- There is no obvious answer
- Maybe let the goal-to-goal direction determine

Clockwise or Counter-Clockwise?



The inner product of two vectors

$$\langle \mathbf{v}, \mathbf{w} \rangle = \mathbf{v}^T \mathbf{w} = \|\mathbf{v}\| \|\mathbf{w}\| \cos(\angle(\mathbf{v}, \mathbf{w}))$$

is positive when the angle between the vector is less than $\frac{\pi}{2}$. Therefore:

$$\langle \mathbf{u}_{GTG}, \mathbf{u}_{FW}^c \rangle > 0 \quad \Rightarrow \quad \mathbf{u}_{FW}^c$$

$$\langle \mathbf{u}_{GTG}, \mathbf{u}_{FW}^{cc} \rangle > 0 \quad \Rightarrow \quad \mathbf{u}_{FW}^{cc}$$

Issues

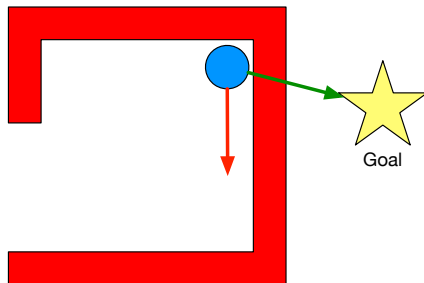
- When release the **follow-wall** behavior?
- How do we decide which behavior to take?

Behaviors revisited

Complete navigation system

A complete navigation system

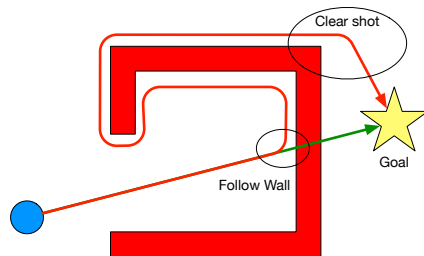
- When do we stop following walls?
- Not when we stop sliding!



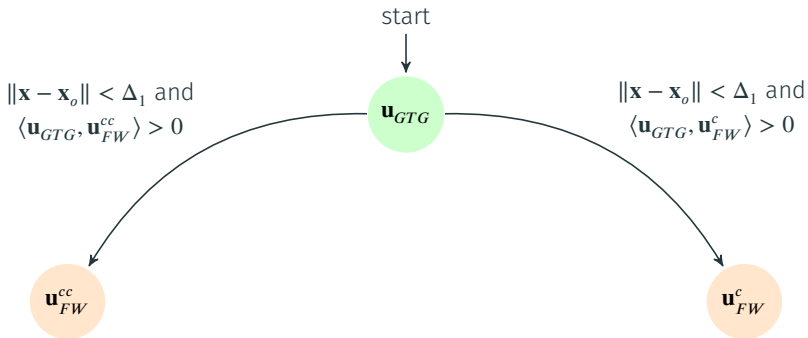
We don't know when to stop sliding...

When to stop?

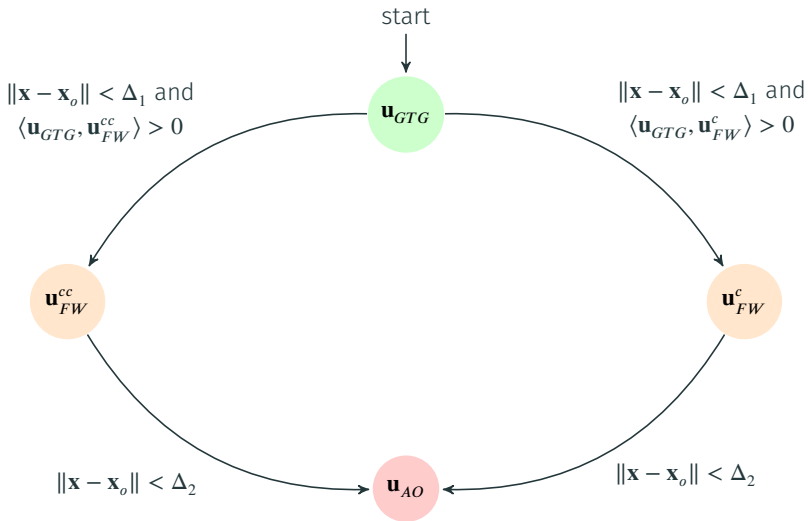
- We should stop when enough progress has been made and we have a “clear shot” to the goal
 - When I am **closer to the goal**
 - You have clear shot towards the goal
- τ is the **time of the last switch**
- **progress:** $\|\mathbf{x} - \mathbf{x}_g\| < \|\mathbf{x}(\tau) - \mathbf{x}_g\|$
- **clear shot:** $\langle \mathbf{u}_{AO}, \mathbf{u}_{GTG} \rangle > 0$ (the angle between the two is less than 90°)



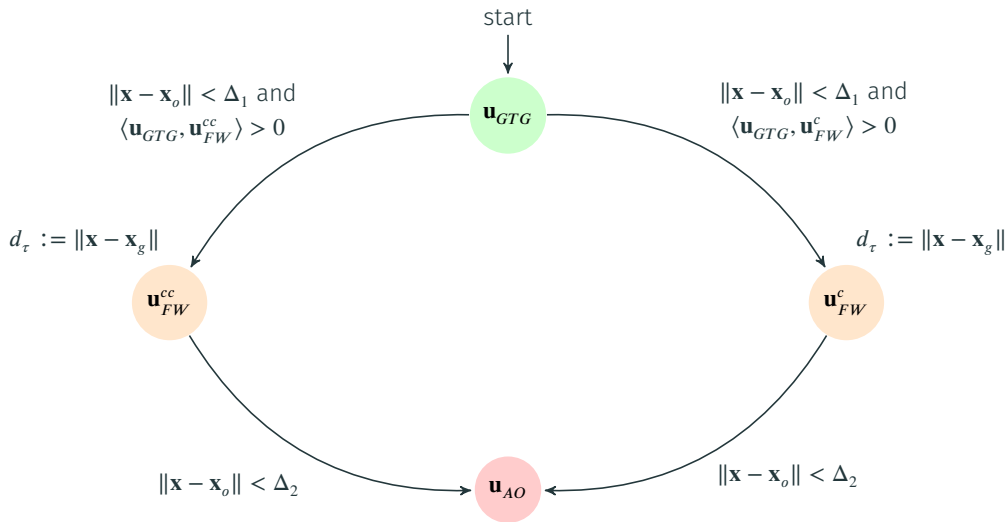
Putting it all together



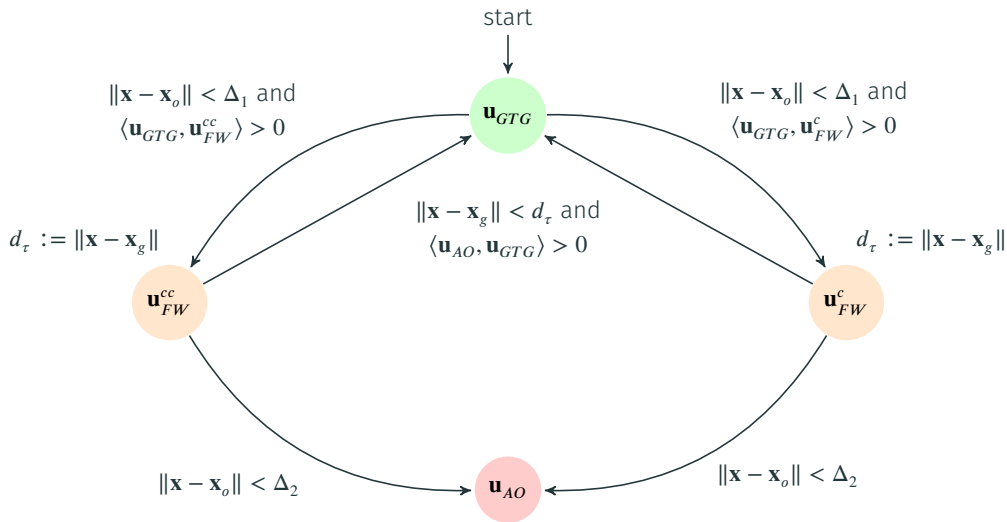
Putting it all together



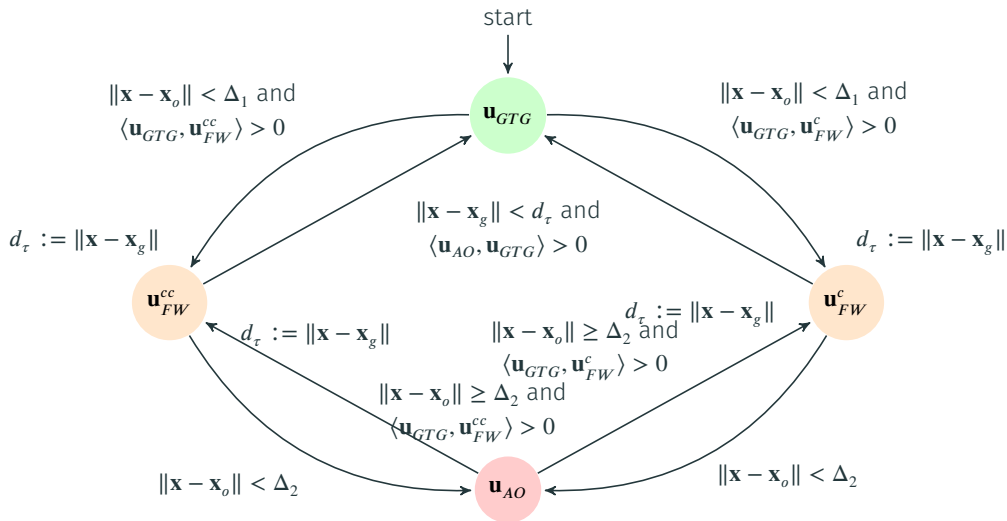
Putting it all together



Putting it all together



Putting it all together



Putting it all together

